A quenched QCD implementation using Fortran coarrays

Lab report - Projektpraktikum, M.Sc. Physik

Nicolas Lang

Abstract A Fortran implementation of quenched QCD using coarrays is presented. The use of derived types, object oriented features of Fortran 2008 and overloaded operators is shown to provide an easy-to-use API. An application to the implementation of the Metropolis algorithm is demonstrated. A sample measurement of the plaquette action using this program agrees with measurements found in literature.

1 Introduction

The evaluation of high-dimensional integrals using probabilistic methods such as Monte-Carlo integration is an essential task in lattice gauge theory. These simulations, carried out on 4-dimensional lattices, are typically computationally expensive. Parallelization and distribution of work across multiple worker nodes is therefore crucial as lattice sizes become larger. The coarray extension now included in the Fortran standard is a convenient tool to implement multi-processing capable applications with data exchange between processes. In this lab, the possibility of such an implementation for the purpose of simulating QCD has been explored. Special attention was given to the development of an easy-to-use API utilizing object-oriented features and operator syntax.

Section 2 briefly discusses the theoretical background of QCD on the lattice. Section 3 introduces Fortran coarrays and describes the general design idea of the implementation. Section 4 discusses the technical details. In section 5, a sample measurement of the plaquette action is presented.

2 Lattice QCD

This section gives a brief introduction to the path integral on the lattice, the QCD action and the Metropolis algorithm. A detailed discussion can be found in [2].

2.1 Path integral quantization on the lattice

The path integral is a quantization method in quantum mechanics and quantum field theory in which the classical principal of stationary action is generalized to quantum-mechanical systems. In the case

Bergische Universität Wuppertal

of a single particle it integrates over all possible paths of motion. In the case of quantum fields it integrates over all possible field configurations. The paths or field configurations are weighted by a phase factor that depends on the action. The path integral for a field Φ is given by

$$Z = \int \mathcal{D}\Phi e^{iS[\Phi]} \tag{1}$$

The weight factor ensures that non-classical paths cancel in the classical limit. For all relevant cases the path integral cannot be computed analytically. Numerical evaluation is possible however by moving from Minkovski space to euclidean space and by discretizing space-time to obtain a finite number of degrees of freedom. On a four-dimensional lattice the path integral takes the form

$$Z = \int \mathcal{D}[\Phi] e^{-S_E[\Phi]} \tag{2}$$

where the measure

$$\mathcal{D}[\Phi] = \prod_{n \in \Lambda} d\Phi(n) \tag{3}$$

is a product of the integration measures of the (classical) field variables located at the lattice sites $n \in \Lambda$ and $S_E[\Phi]$ is called the euclidean action. $e^{-S_E[\Phi]}$ is called Boltzmann factor and is a measure for the probability of a given configuration. Hence the path integral is analogous to a partition function of statistical mechanics.

The expectation value of an operator O can be obtained from

$$\langle O \rangle = \frac{1}{Z} \int \mathcal{D}[\Phi] e^{-S_E[\Phi]} O[\Phi] \tag{4}$$

. i.e. the operator becomes a functional of the field variables and its expectation value is the result of its integration over all configurations weighted by the Boltzmann factor.

2.2 The QCD action

This lab report discusses the implementation of the quenched approximation of quantum chromodynamics (QCD), i.e. QCD without dynamical quarks. Therefore only the gluon action will be discussed. The euclidean gluon action in the continuous case is given by

$$S_G[A] = \frac{1}{2g^2} \int d^4x \ \mathrm{tr}[F_{\mu\nu}(x) F_{\mu\nu}(x)] \tag{5}$$

where A is the gauge field and $F_{\mu\nu}(x)$ is the value of the gluonic field strength tensor at x. By introducing link variables $U_\mu(n)$ (which are elements of SU(3)) on the links between the lattice sites, a discretized version of the gluon action

$$S_G[U] = \frac{\beta}{3} \sum_{n \in \Lambda} \sum_{\mu < \nu} \Re \ \mathrm{tr}[1 - U_{\mu\nu}(n)] \tag{6}$$

can be derived, where

$$\beta = \frac{6}{g^2} \tag{7}$$

is referred to as *inverse coupling* and

$$U_{\mu\nu}(n) = U_\mu(n)U_\nu(n+\hat{\mu})U_\mu(n+\hat{\nu})^\dagger U_\nu(n)^\dagger \tag{8}$$

is called *plaquette*. The plaquette represents the smallest non-trivial loop of link variables and its trace is a gauge invariant quantity.

2.3 Monte-Carlo integration and the Metropolis algorithm

In order to compute the vacuum expectation value of an operator O of the QCD gauge field on a lattice, the integral

$$\langle O \rangle = \frac{1}{Z} \int \mathcal{D}[U] e^{-S_G[U]} O[U] \tag{9}$$

has to be evaluated. Since

$$\mathcal{D}[U] = \prod_{n \in \Lambda} \prod_{\mu=1}^{4} dU_\mu(n) \tag{10}$$

this clearly is a computationally impossible endeavor even for moderately large lattices. The Monte-Carlo method provides a way to approximate the integral by averaging over a randomly generated sample of possible function values of the integrand. The integral then turns into a sum over the sample of generated configurations. It is favorable to generate the configurations according to the distribution given by the Boltzmann factor. The probability distribution density for the sampling becomes

$$dP(U) = \frac{e^{-S[U]}\mathcal{D}[U]}{\int \mathcal{D}[U] e^{-S[U]}} \tag{11}$$

. The expectation value of an operator is then given by

$$\langle O \rangle = \lim_{N \to \infty} \frac{1}{N} \sum_{n=1}^{N} O[U_n] \tag{12}$$

where N is the sample size. This is called importance sampling. Configurations distributed according to this distribution density can be generated using the Metropolis algorithm. This algorithm implements a Markov process in which configurations are updated iteratively, starting from an arbitrary initial configuration, to eventually evolve to an equilibrium distribution that follows the desired probability density. Observables should be measured only after a sufficient number of equilibrating steps has been performed. It should be noted that it is important, that the Markov process can reach every point in configuration space. The Metropolis algorithm includes the following steps:

1. Given a field configuration, choose a site n and direction μ and generate a random link update $U_\mu(n)' \in SU(3)$
2. Compute the action change ΔS. Since the action has been identified with the plaquette observable (see equation 6), ΔS will only depend on the six plaquettes that share the updated link. It is convenient to first compute the products over the other 3 gauge links that form the plaquette together with the updated link. These products are called staples and the sum over all staples is given by

$$A = \sum_{i=1}^{6} P_i = \sum_{\nu \neq \mu} \big(U_\nu(n+\hat{\mu}) U_\mu(n+\hat{\nu})^\dagger U_\nu(n)^\dagger \tag{13}$$

$$+ U_\nu(n+\hat{\mu}-\hat{\nu})^\dagger U_\mu(n-\hat{\nu})^\dagger U_\nu(n-\hat{\nu}) \big) \tag{14}$$

The action change becomes

$$\Delta S = -\frac{\beta}{N} \Re \text{ tr}[(U_\mu(n)' - U_\mu(n))A] \tag{15}$$

The new link $U_\mu(n)'$ should be close to the old link. Therefore new links are commonly generated by multiplying the old link with an update matrix $X \in SU(3)$ which is close to unity. A method to generate such update matrices is described in [2].

3. Generate a real random number r uniformly distributed between 0 and 1. Accept the link update if $r \leq \exp(-\Delta S)$. Otherwise reject it.
4. Repeat these steps with a new link

3 Overview and general concept

After the physical background and general idea of lattice QCD has been introduced, the next sections will focus on the implementation of a QCD simulation in Fortran. Only the simpler case of static quarks (quenched approximation) has been implemented. The focus on the technical side lies on data parallelism leveraging Fortran coarrays. This section will discuss the general design ideas of the implementation.

3.1 Fortran coarrays and shortcomings

Coarrays provide a way to distribute data across multiple program instances while retaining a concise syntax that abstracts the underlying inter-process or network communication. Formerly an extension of Fortran 95, coarrays are now part of the Fortran 2008 standard. Coarrays are variables with one or several *codimensions* and indices referencing data stored by another program instance, called *image*. *Images* represent copies of the program that run asynchronously and can possess further independent local variables. The coarray entries can be accessed from every image by specifying the respective *coindex* representing the image, where the data is stored. The size of the codimension is determined at runtime from the number of running images. The standard provides for additional methods that enable the user to obtain the number of images and the index of the current image and to introduce a break point to synchronize images. The coarray support in the GNU Fortran compiler is based on the *OpenCoarrays* [1] library which in turn is based on the Message Passing Interface (MPI).

The current implementation of coarrays in Fortran suffers from a number of shortcomings. Most notably, there is no notion of coarray pointers. This makes data encapsulation difficult. A new proposal called Coarray Fortran 2.0 [3] addresses this and other issues, but it is not clear whether and when it will become part of the Fortran standard. Therefore, work-arounds need to be found.

3.2 Design choices

The goal of this implementation of quenched QCD is to provide an easy-to-use API, where the actual storage and distribution of data across images is mainly hidden from the user. Types (classes) that offer a global view on fields and field variables will be implemented, i.e. operations will be performed on the field as a whole and not individual lattice sites. To work with operators depending on nearest neighbors, shift operators will be provided. To overcome limitations associated with coarrays, such as prohibited coarray return values, local versions of certain types will be used. The gauge field itself will be implemented as a wrapper around a global data variable (with the assumption that only ever one field instance will be needed).

4 Implementation

This section is dedicated to the particular implementation of the aforementioned ideas. The derived types and operators are discussed first. Then a concise implementation of the metropolis algorithm is presented.

4.1 Types

A number of derived types is needed in order to hide underlying data structures from the user and to facilitate global field operations. These types can be grouped into field types, which keep a record of spatial and temporal indices, and field variable types, which are meaningful only with respect to a given field or in the context of reductions. The implementation makes use of object oriented features introduced in Fortran 2003.

Gauge field

The *gauge_field* type serves as an auxiliary handle to perform operations on the gauge field itself. It does not contain any data as a member but instead accesses the global module variable *data* from its type bound methods. The following code snippet shows the type definition:

Listing 1: *Type definition of the* gauge_field *type*

```
complex, allocatable :: data(:,:,:)[:]

type, public :: gauge_field
    integer, allocatable :: indices(:,:,:,:,:)
    logical :: tmp
    integer :: dir
    integer :: par
    integer :: x_low, x_up, y_low, y_up, z_low, z_up, t_low, t_up
    integer :: nx, ny, nz, nt
contains

    ! ...
    procedure :: initialize
    procedure :: update
    ! ...

end type gauge_field
```

Data is stored linearly. The *data* variable is an allocatable coarray of type complex with three dimensions and one codimension. It's first two dimensions are those of the complex matrices, running from 1 to 3 in the case of SU(3). The third dimension is the local index. The codimension (in square brackets) represents the index of the image. These indices will be refered to as *storage indices*. As field operators will often depend on link variables of the nearest neighbours, performing parallel operations requires lattice sites to be divided into sites of even or odd parity. Parity is defined by the euclidean indices as $p = (x + y + z + t)\ mod\ 2$. Thus, all sites of a given parity can be updated in parallel without affecting one another. It is then favourable from a performance perspective to align data of a given parity in memory in order to avoid cache misses. Data is therefore organized in the order $x, y, z, t, direction, parity$, i.e. link variables of a particular direction and parity that are next to each other in x are stored consecutively. Euclidean indices and link direction will be referred to as *field indices*. The translation between field indices and storage indices is done with the help of an index array. Its dimensions represent in this order: index/coindex, $x, y, z, t, direction$. Parity is not stored

as an index since it can be calculated from the euclidean indices. The *gauge_field* type possesses two integer members *dir* and *par* which serve as flags on temporaries used in arithmetic operations. This is further explained when discussing operators. The integers x_{low}, x_{up} etc. define the boundaries of the current image, n_x, ..., n_t the lengths of the respective dimensions. The *initialize* method allocates the data array and initializes the matrices either to the unit matrix or random unitary 3×3 matrices. It also fills the index array and distributes the data to the available images. The latter is performed in the t and z dimensions. The *update* method updates all link variables of parity *pin* and direction *din* with values stored in the *su3_field_var* variable supplied as the third argument, if the corresponding link value stored in the fourth argument of type *logical_field_var* is set to .true..

Listing 2: *Update subroutine of the* gauge_field *type. The subroutine* comp_dimensions *computes* d_low, d_up *and* step_size *from the values of the direction and parity flags*

```
subroutine update(a, pin, din, fv, lfv, acc)
    class(gauge_field), intent(inout) :: a
    integer, intent(in) :: pin, din
    type(loc_su3_field_var), intent(inout) :: fv
    type(loc_logical_field_var), intent(in) :: lfv
    integer, intent(inout) :: acc
    integer :: i,j,k,l,d, idx
    integer :: d_low, d_up, da, la, step_size

    a%par = pin
    a%dir = din

    call check_dim_fv(a, size(fv%data)/9)
    call check_dim_fv(a, size(lfv%data))
    call comp_dimensions(a, fv, d_low, d_up, step_size)

    idx = 1
    do d = d_low, d_up                          ! directions
        do i = a%t_low, a%t_up                  ! t
            do j = a%z_low, a%z_up              ! z
                do k = a%y_low, a%y_up                                  ! y
                    do l = a%x_low-1+step_size, a%x_up, step_size       ! x
                        if (d == 0) then                ! specific direction
                            da = abs(a%dir)
                        else                            ! all directions
                            da = d
                        end if
                        if (step_size == 2) then        ! defined parity
                            la = l-abs(mod(i+j+k,2)-a%par)
                        else                            ! undefined parity
                            la = l
                        end if

                        if (lfv%data(idx)) then
                            if (fv%conj) then
                                data(:,:,a%indices(0,la,k,j,i,da)) &
                                [a%indices(1,la,k,j,i,da)] &
                                = conjg(transpose(fv%data(:,:,idx)))
                            else
                                data(:,:,a%indices(0,la,k,j,i,da)) &
                                [a%indices(1,la,k,j,i,da)] &
                                = fv%data(:,:,idx)
                            end if
                            acc = acc + 1
                        end if
                        idx = idx + 1

                    end do
```

```
                end do
            end do
        end do
    end do

    ! reset parity and direction again
    a%par = -1
    a%dir = 0

end subroutine
```

Field variables

Field variable types possess data members which are either arrays or coarrays of a given type and represent the results of operations on *gauge_fields* and other field variables. They do not have any notion of spatial and temporal indices or direction. This is intended since the operands of a binary operator creating the field variable could have different parities, directions or shifts. Therefore the indices of the result would not be well defined. Hence, field variables are only meaningful in operations with gauge fields or reductions, such as sums or mean values. Currently 3 types of field variables are implemented:

- SU(3)-matrix-valued: *su3_field_var*
- real-valued: *real_field_var*
- logical-valued: *logical_field_var*

Listing 3: *Type definition of the* su3_field_var *type*

```
type, public :: su3_field_var
    complex, allocatable :: data(:,:,:)[:]
    logical :: conj = .false.
contains
    procedure :: add
    procedure :: add_glob_loc
    procedure :: assign
    procedure :: increment
    procedure :: real_trace
    procedure :: init_rnd
    procedure :: init_zero
    generic :: operator (+) => add, add_glob_loc
    generic :: assignment (=) => assign
    generic :: operator (.rtrc.) => real_trace
end type su3_field_var
```

Additionally, each field variable type comes in a local version, meaning that the data member is an array instead of a coarray. This is necessary since derived types containing coarrays cannot be function return values. An overloaded assignment operator on the (global) field variable types allows the assignment of local field variables to (global) field variables, i.e. the array is copied into the local part of the coarray on every image. Reductions are only possible on (global) field variables for obvious reasons. Increment and initialization routines are defined for the *su3_field_var* and *real_field_var* types. The .rtrc. operator calculates the real part of the trace on an *su3_field_var* and returns a *real_field_var*.

4.2 Operators

For convenient use of the *gauge_field* type and the field variable types in arithmetic operations and comparisons, various operators are overloaded. On *gauge_field*s additional select and shift operators are defined to specify which links should be used in subsequent operations.

Select and shift

As most operations are not intended to be applied to every link simultaneously select and shift operators are defined on *gauge_field*s. The select operator enables the user to specify a parity and a link direction. It is a binary operator where one of the operands is a *gauge_field* and the other an integer array with two entries, the first of which specifies parity and the second the link direction. If both parities are needed a value of -1 is used for the first entry. To select all directions the second entry is set to 0. Negative directions cause the respective links to be conjugated in the subsequent operation. The select operator creates a temporary and sets the parity and direction flags on it to the respective values.

Listing 4: *Select operator of the* gauge_field *type*

```
function select (a, b) result (tmp)
    class(gauge_field), intent(in) :: a
    integer, dimension(2), intent(in) :: b
    type(gauge_field) :: tmp

    call a%copy_shift(tmp, 0, 0)
    tmp%par = b(1)
    tmp%dir = b(2)
    tmp%tmp = .true.
end function
```

The shift operator is used to select neighboring sites (as needed by gauge invariant observables such as the plaquette). It is a binary operator whose operands are a *gauge_field* and an integer array with two entries. The first entry represents the shift direction, the second entry the shift distance. The shift operator returns a temporary copy of the current *gauge_field* where the index array has been shifted in the specified direction and by the specified distance.

Listing 5: *Shift operator of the* gauge_field *type*

```
function shift (a, b) result (tmp)
    class(gauge_field), intent(in) :: a
    integer, dimension(2), intent(in) :: b
    type(gauge_field) :: tmp

    call a%copy_shift(tmp, b(1), b(2))
    tmp%tmp = .true.
end function
```

Arithmetic and comparison operators

Arithmetic and comparison operators can be defined between two *gauge_field*s, between a *gauge_field* and an *su3_field_var* or between field variables. The abundance of types and possible combinations requires a large number of operator implementations, many of which only differ in the declaration section of the function. Since Fortran does not provide templates, code duplication is unavoidable. The function body mostly contains a nested do-loop running over all euclidean dimensions x, y, z, t

and all directions, if the direction flag is set to 0. Otherwise, only the selected direction is considered. If the parity flag is set to either 0 or 1, the variable *step_size* is set to 2 such that every other lattice site is skipped. The x-index l gets redefined as la = l−abs(mod(i+j+k,2)−a%par) according to the selected parity, where a is the *gauge_field* and k, j, i index the y-, z- and t-dimensions. Currently only those operators needed for the plaquette/staple are implemented. Since it is always possible to convert between local and global field variables, most of them are only available for the local versions of the field variables.

Listing 6: *Multiplication operator between a* gauge_field *and an* su3_field_var. *The function multSu3 is a wrapper around the Basic Linear Algebra Subroutines (BLAS) function cgemm. The second and fourth argument indicate whether the matrix should be conjugated.*

```
function multiply_fv(a, b) result (res)

    class(gauge_field), intent(in) :: a
    type(su3_field_var), intent(in) :: b
    type(loc_su3_field_var) :: res
    integer :: i,j,k,l,d, idx
    integer :: d_low, d_up, da, la, step_size

    call comp_dimensions(a, res, d_low, d_up, step_size)
    call check_dim_fv(a, size(b%data)/9)

    idx = 1
    do d = d_low, d_up
        do i = a%t_low, a%t_up
            do j = a%z_low, a%z_up
                do k = a%y_low, a%y_up
                    do l = a%x_low-1+step_size, a%x_up, step_size
                        if (d == 0) then
                            da = abs(a%dir)
                        else
                            da = d
                        end if
                        if (step_size == 2) then
                            la = l-abs(mod(i+j+k,2)-a%par)
                        else
                            la = l
                        end if

                        res%data(:,:,idx) = multSu3( &
                        data( :,:,a%indices(0,la,k,j,i,da)) &
                        [a%indices(1,la,k,j,i,da)], &
                        (a%dir < 0), &
                        b%data(:,:,idx), &
                        b%conj )
                        idx = idx + 1

                    end do
                end do
            end do
        end do
    end do

end function
```

It is necessary that operands have the same field dimensions, i.e. if odd parity has been selected on one of the operands then parity (but not necessarily the same) needs to be set on the other. In the simpler case of operators between two field variables, a single do-loop iterates over the data array. For addition/subtraction and comparison operators, the intrinsic operators on arrays are used. Which operators are currently implemented for which types can be seen in table 1. Comparison operators

Table 1: *Arithmetic and comparison operators and possible operands*

Operator	Types
add (+)	*gauge_field*, *su3_field_var*, *real_field_var*
subtract (-)	*gauge_field*, *su3_field_var*, *real_field_var*
multiply (*)	*gauge_field*, *su3_field_var*, scalars
less than (<)	*real_field_var*
less than or equal (<=)	*real_field_var*
equal (==)	*real_field_var*
greater than or equal (>=)	*real_field_var*
greater than (>)	*real_field_var*

produce *logical_field_var*s as their results. To facilitate operations including intrinsic types like scalars and to overload intrinsic functions like *exp* for field variables, an interface module is provided.

4.3 The metropolis module

The metropolis module provides a concise implementation of the metropolis algorithm using the types and operators described above. It contains the *staple* function and the *metro* subroutine as well as an array of 100 *su3_field_var*s containing update matrices close to unity. The *init_update_tab* subroutine allocates the update table. Matrices close to unity are produced from SU(2) generators following the method described in [2]. The staple function makes use of the *increment* method of *su3_field_var*s to avoid unnecessary temporaries.

Listing 7: *Staple function*

```
function staple(u, p, d)

    type(gauge_field), intent(in) :: u
    integer, intent(in) :: d, p
    integer :: i, s
    type(loc_su3_field_var) :: staple

    s = u%nx * u%ny * u%nz * u%nt / 2

    call staple%init_zero(s)
    do i = 1, 4
        if (i /= d) then
            call staple%increment( &
                (u.sel.[p,i]) &
                * (u.shift.[i,1].sel.[p,d]) &
                * (u.shift.[d,1].sel.[p,-i]) &
                + (u.shift.[i,-1].sel.[p,-i]) &
                * (u.shift.[i,-1].sel.[p,d]) &
                * (u.shift.[i,-1].shift.[d,1].sel.[p,i]) &
                )
        end if
    end do

end function staple
```

The metropolis routine initializes the update table based on the value of *eps* if it has not been initialized yet and then iterates over even and odd sites and all directions. In every iteration a candidate link *su3_field_variable* is produced from the gauge field and a randomly chosen field variable from the update table. The selected update field variable is conjugated in 50% of the cases to fulfill the detailed balance criterion. The action change $delta_s$ is represented by a *real_field_var* and calculated from the

old and new links making use of the real trace operator .rtrc.. Finally the *gauge_field*'s update routine is called using a randomly initialized *real_field_var* and the exponentiated action delta for the update condition, the former being required to be smaller or equal to the latter for an update to take place. The metropolis routine will also keep track of the acceptance rate.

Listing 8: *Metro subroutine running one sweep for both parities and each direction*

```
subroutine metro(u, eps, beta, acc)

    type(gauge_field), intent(inout) :: u
    real, intent(inout) :: acc
    real, intent(in) :: eps, beta
    type(loc_su3_field_var) :: stp, new_link
    type(loc_real_field_var) :: delta_s, rnd_field
    integer :: p, d, s, acc_count
    real :: rnd1, rnd2

    s = u%nx * u%ny * u%nz * u%nt / 2
    acc_count = 0

    ! generate link updates
    call init_update_tab(100, s, eps)

    ! iterate over both parities and all directions
    do p = 1,2
        do d = 1,4

        ! calc. staple
        stp = staple(u,p,d)
        stp%conj = .true.

        ! generate new link
        call random_number(rnd1)
        call random_number(rnd2)
        if (rnd1 >= 0.5) then
            update_tab(ceiling(rnd2*100))%conj = .true.
        else
            update_tab(ceiling(rnd2*100))%conj = .false.
        end if

        new_link = (u.sel.[p,d]) * update_tab(ceiling(rnd2*100))

        ! calculate action change
        delta_s = -beta / 3.0 * .rtrc.((new_link - (u.sel.[p,d])) * stp)

        ! accept with some prob. dep. on action
        call rnd_field%init_rnd(s)
        call u%update(p, d, new_link, (rnd_field <= exp(-delta_s)), acc_count)
        sync all

        end do
    end do

    acc = real(acc_count) / real((s*8))

end subroutine metro
```

5 Sample measurement

This last section briefly presents a sample measurement of the plaquette expectation value as a proof of concept. Fig. 1 shows the normalized plaquette action

$$E_P = \frac{\langle S_P \rangle}{6|\Lambda|} = \frac{1}{6|\Lambda| \cdot 3} \sum_{n \in \Lambda} \sum_{\mu < \nu} \Re \operatorname{tr}[U_{\mu\nu}] \tag{16}$$

plotted against β. The plot has been generated on a 16^4 lattice, starting from a configuration of unit matrices, increasing β in steps of 0.2 and using 100 equilibrating updates after every change, followed by 100 groups of two updates and one measurement. This amounts to a total of 12000 updates and 4000 measurements. The simulation ran on 4 cores of an Intel i7 2600k and took 4.41 hours to complete. The plot agrees with other measurements found in literature (e.g. in [2]).

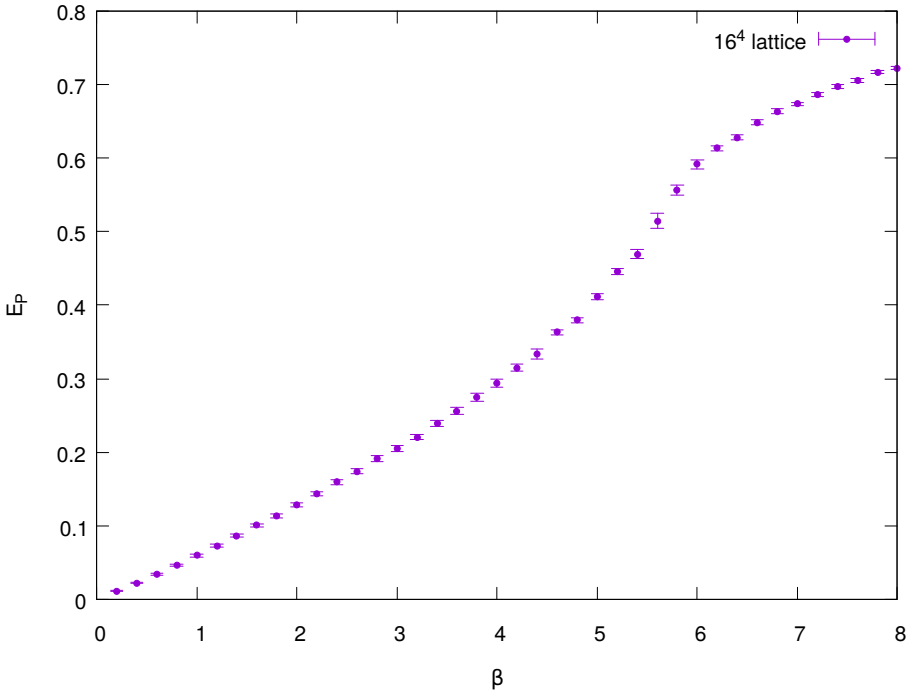


Fig. 1: *The plaquette action E_P plotted against β on a 16^4 lattice. The data points represent the mean value and the error bars the standard deviation obtained from 100 measurements per value of β.*

6 Summary

In this lab, an implementation of the quenched approximation of QCD on the lattice using Fortran coarrays has been developed. The implemented API makes use of Fortran's object oriented features and overloaded operators and provides the user with easy-to-use types representing fields and field variables. Using these types an implementation of the Metropolis algorithm has been shown. A sample measurement of the plaquette action on a 16^4 lattice terminated after 4.41 hours on an Intel i7 2600k

using 4 cores performing 12000 update sweeps and 4000 measurements. The resulting plot showing a β-scan of the plaquette action agrees with measurements found in literature.

References

[1] Alessandro Fanfarillo et al. “OpenCoarrays: Open-source Transport Layers Supporting Coarray Fortran Compilers”. In: *Proceedings of the 8th International Conference on Partitioned Global Address Space Programming Models*. ACM, Oct. 2014, 4.

[2] Christof Gattringer et al. *Quantum Chromodynamics on the Lattice*. 2010. ISBN: 978-3-642-01849-7. DOI: 10.1007/978-3-642-01850-3.

[3] John Mellor-Crummey et al. “A New Vision for Coarray Fortran”. In: *Proceedings of the Third Conference on Partitioned Global Address Space Programing Models*. PGAS ’09. Ashburn, Virginia, USA: ACM, 2009, 5:1–5:9. ISBN: 978-1-60558-836-0. DOI: 10.1145/1809961.1809969.